

OAuth & mailbox credentials — setup guide

How to create the sign-in and mailbox-connect credentials EmailWarm needs for Google, Microsoft, Zoho and Yahoo — set up once at your webnco.xyz org and reused across your development apps. Every redirect URL, scope and environment variable here was taken straight from the app's code.

Audience: whoever configures the OAuth apps (you, as developer/admin) · Verified against the EmailWarm codebase

Contents

1. Which platform needs what
 2. Before you start
 3. Google — one org-level OAuth app
 4. Microsoft — Azure app registration
 5. Zoho — app password
 6. Yahoo — app password
 7. Where every value goes (env)
 8. Test it works & fix errors
 9. Reusing one app across many projects
-

PART 1 Which platform needs what

EmailWarm uses two different kinds of credential, and it matters which one each platform needs. Read this table first — it decides the rest of the guide.

PLATFORM	HOW EMAILWARM CONNECTS IT	WHAT YOU CREATE	COVERED IN
Google (Gmail, Workspace)	OAuth — "Connect with Google" button and "Sign in with Google"	One Google Cloud OAuth client (ID + secret)	Part 3
Microsoft (Outlook, 365)	OAuth — "Connect with Microsoft" and "Sign in with Microsoft"	One Azure (Entra ID) app registration (ID + secret)	Part 4
Zoho	Custom SMTP / IMAP form — no OAuth	A Zoho app-specific password	Part 5
Yahoo	Custom SMTP / IMAP form — no OAuth	A Yahoo app password	Part 6
Any other host	Custom SMTP / IMAP form — no OAuth	Mailbox password or the host's app password	Part 6

THE SHORT VERSION

You only build **OAuth apps for Google and Microsoft**. Zoho, Yahoo and everything else connect through the app's Custom SMTP / IMAP form with an app password, so they need no console setup at all — you can skip straight to Part 5 for those. The OAuth apps are the only reusable, org-level thing, which is exactly what you're planning for webnco.xyz.

WHAT EACH OAUTH APP POWERS IN EMAILWARM

- **Sign in with Google / Microsoft** — logging into EmailWarm itself (handled by the auth library at `/api/auth/callback/<provider>`).
- **Connect a mailbox** — one-click connect on the Accounts page (handled at `/api/mailbox-oauth/callback/<provider>`).

Both live under the same OAuth app, so each provider needs **two redirect URLs** registered. That is the single most common thing people get wrong, so the exact URLs are called out in each part.

PART 2 Before you start

1 Decide your base URL

Every redirect URL is built from the address EmailWarm runs on. For local development that is `http://localhost:3000`. If you later run it somewhere else (a staging domain), you'll register the same paths under that address too. This guide uses `http://localhost:3000` throughout; swap in your real address where needed.

2 Know where the values will go

Each client ID and secret is pasted into **two files** — the backend's `backend/.env` and the frontend's `frontend/.env.local` — with the **same values in both**. The full list is in Part 7. Keep a scratch note open to collect the four values (Google ID, Google secret, Microsoft ID, Microsoft secret) as you create them.

TREAT SECRETS LIKE PASSWORDS

Client secrets and app passwords grant access to accounts. Keep them only in `.env` files (which are git-ignored), never in code, screenshots, chat or this document. If one leaks, rotate it in the provider's console.

PART 3 Google — one org-level OAuth app OAUTH

You'll create a single OAuth client under your webnco.xyz Google account and reuse it for EmailWarm and future dev apps. Do this at console.cloud.google.com, signed in as a webnco.xyz user.

1 Create (or pick) a project

Top bar → project picker → **New Project**. Name it something reusable like "WebNCO Dev" and, if webnco.xyz is a Google Workspace org, set its **Organization / Location** to webnco.xyz. One project can hold the OAuth client for many apps.

2 Choose the consent-screen audience IMPORTANT DECISION

Go to **APIs & Services** → **OAuth consent screen**. You pick one of two user types, and the choice has real consequences because EmailWarm asks for Gmail's **restricted** full-mail scope:

CHOICE	WHO CAN CONNECT	TRADE-OFF
Internal (Workspace only)	Only webnco.xyz mailboxes	No Google verification needed, no test-user cap, and refresh tokens don't expire. Best if you mainly warm webnco.xyz addresses.
External + "Testing"	Any Google account you add as a test user (up to 100)	Works for personal Gmail too, but shows an "unverified app" warning, and refresh tokens expire after 7 days , so test mailboxes need reconnecting weekly.

RECOMMENDATION FOR WEBNCO.XYZ

If webnco.xyz is a Workspace org and you'll warm webnco.xyz mailboxes, choose **Internal** — it's simpler and avoids the 7-day token expiry. To also connect personal Gmail addresses for testing, either add a second **External** client later, or just connect those through the Custom SMTP form with an app password (Part 5). Going fully public (External + "In production") requires Google's restricted-scope security assessment, which is slow and costly — not needed for development.

3 Fill the consent screen & add the scope

Enter an app name ("EmailWarm Dev"), your support email and a developer contact. On the Scopes step, add this exact scope (plus the basic `openid` and `email`, usually added for you):

```
https://mail.google.com/      # full Gmail access (send + IMAP)
openid
email
```

If you chose **External + Testing**, add every Gmail address you'll connect as a **Test user** on the next step.

4 Create the OAuth client and register both redirect URLs

Go to **APIs & Services** → **Credentials** → **Create Credentials** → **OAuth client ID**. Application type: **Web application**. Name it "EmailWarm local". Under **Authorized redirect URIs**, add **both** of these exactly:

```
http://localhost:3000/api/auth/callback/google # sign in
http://localhost:3000/api/mailbox-oauth/callback/google # connect mailbox
```

You can leave Authorized JavaScript origins empty. Click Create.

WHAT YOU GET

A dialog with a **Client ID** (ends in `.apps.googleusercontent.com`) and a **Client secret**. Copy both into your scratch note. You can reopen them any time from the Credentials list.

5 Enable the Gmail API

APIs & Services → **Library** → search "Gmail API" → **Enable**. (The scope grants access; the API must also be switched on for the project.)

6 Put the values in the env files

backend/.env and frontend/.env.local (same values in both)

```
GOOGLE_CLIENT_ID=xxxxxxx.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=GOCSPX-xxxxxxxxxxxxxxxxxx
```

Restart both the backend and frontend after editing. See Part 7 for the complete file layout.

PART 4 Microsoft — Azure app registration OAUTH

The Microsoft equivalent is an "app registration" in the Microsoft Entra admin centre (formerly Azure AD). Go to entra.microsoft.com → **Applications** → **App registrations** → **New registration**.

1 Register the app

- **Name:** "EmailWarm Dev".
- **Supported account types:** choose "**Accounts in any organizational directory and personal Microsoft accounts**". This lets both company 365 mailboxes and personal Outlook/Hotmail accounts connect. (The app uses the `common` tenant, which matches this choice.)
- **Redirect URI:** platform **Web**, and add the first URL below now. You'll add the second in step 2.

```
http://localhost:3000/api/auth/callback/microsoft      # sign in
http://localhost:3000/api/mailbox-oauth/callback/microsoft # connect mailbox
```

2 Add the second redirect URL

Open the new app → **Authentication** → under the Web platform, **Add URI**, and add the mailbox-connect URL from above so both are listed. Save.

3 Add the API permissions (scopes)

API permissions → **Add a permission**. EmailWarm uses Outlook's own mail endpoints, not Microsoft Graph, so add these **delegated** permissions:

```
openid
email
offline_access
https://outlook.office.com/IMAP.AccessAsUser.All # read the mailbox
https://outlook.office.com/SMTP.Send             # send mail
```

In the console, the two Office permissions are listed under the "**Office 365 Exchange Online**" API (search it under "APIs my organization uses" if it's not on the first list). The first three are standard delegated permissions. No admin consent is needed for personal accounts; a company tenant may require an admin to approve.

4 Create a client secret

Certificates & secrets → New client secret. Give it a description and an expiry (24 months is fine for dev). Copy the secret's **Value** immediately — it's shown only once. The app's **Application (client) ID** is on the Overview page.

WHAT YOU GET

An **Application (client) ID** (a GUID) and a **client secret value**. Copy both to your scratch note.

5 Put the values in the env files

backend/.env and frontend/.env.local (same values in both)

```
MICROSOFT_CLIENT_ID=00000000-0000-0000-0000-000000000000  
MICROSOFT_CLIENT_SECRET=xxxxxxxxxxxxxxxxxxxxxx
```

COMMON MICROSOFT BLOCKER

Many company 365 tenants disable "SMTP AUTH" and third-party IMAP by policy. If a company Outlook mailbox refuses to send even with correct OAuth, an admin must allow authenticated SMTP for that mailbox. Personal Outlook/Hotmail accounts don't have this restriction.

PART 5 Zoho — app password APP PASSWORD

EmailWarm has no Zoho OAuth; Zoho connects through the **Custom SMTP / IMAP** form. You only need an app-specific password.

1 Create the app password

1. Sign in to Zoho Mail, go to `accounts.zoho.com` → **Security** → **App Passwords**.
2. Turn on two-factor authentication first if prompted.
3. Click **Generate New Password**, name it "EmailWarm", and copy the code.

2 Use it in EmailWarm

On the Accounts page, choose **Custom SMTP / IMAP** (or the pool wizard's **Custom** type), press the **Zoho** quick-fill, and enter the app password as both the SMTP and IMAP password. Servers:

`smtp.zoho.com:587` and `imap.zoho.com:993` .

CUSTOM DOMAINS ON ZOHO

If your `webnco.xyz` mail is hosted on Zoho, this is also how you connect a `@webnco.xyz` mailbox — Custom form, Zoho servers, Zoho app password. No OAuth app involved.

1 Yahoo

1. Yahoo → **Account Info** → **Account Security**; turn on two-step verification.
2. Click **Generate app password**, choose "Other app", name it "EmailWarm", copy the code.
3. In EmailWarm's Custom form, use the **Yahoo** quick-fill: `smtp.mail.yahoo.com:587` and `imap.mail.yahoo.com:993`, with the app password in both password fields.

2 Gmail / Outlook without OAuth ALTERNATIVE

You don't have to use OAuth for Google or Microsoft. If you'd rather skip the console setup for a quick test, both also work through the Custom form with an app password (Gmail: `myaccount.google.com/apppasswords`; Outlook: app password under Microsoft account security). This is the path the end-user Getting Started guide uses. OAuth is the nicer one-click experience; app passwords are faster to set up for a developer.

3 Any other host (GoDaddy, cPanel, custom)

Get the SMTP and IMAP server, port, username and password from the host (usually under "email client setup" / "IMAP settings"). Enter them in the Custom form. Outgoing is normally port 587, incoming 993.

PART 7 Where every value goes

The four OAuth values go into both env files, identically. Restart the backend and frontend after any change.

BACKEND/.ENV

```
GOOGLE_CLIENT_ID=xxxxxxx.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=GOCSPX-xxxxxxxxxxxxxxxxxx
MICROSOFT_CLIENT_ID=00000000-0000-0000-0000-000000000000
MICROSOFT_CLIENT_SECRET=xxxxxxxxxxxxxxxxxxxxxxxxxx

# Already set for local dev – leave as is:
APP_URL=http://localhost:3000
```

FRONTEND/.ENV.LOCAL

```
GOOGLE_CLIENT_ID=xxxxxxx.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=GOCSPX-xxxxxxxxxxxxxxxxxx
MICROSOFT_CLIENT_ID=00000000-0000-0000-0000-000000000000
MICROSOFT_CLIENT_SECRET=xxxxxxxxxxxxxxxxxxxxxxxxxx

# Already set for local dev – leave as is:
BETTER_AUTH_URL=http://localhost:3000
NEXT_PUBLIC_BETTER_AUTH_URL=http://localhost:3000
```

IF YOU RUN ON A DIFFERENT ADDRESS

The mailbox-connect redirect is built from `APP_URL`. If EmailWarm ever runs somewhere other than `http://localhost:3000`, either set `APP_URL` to the real address (and register the matching redirect URLs in both consoles), or override just the callback with `GOOGLE_MAILBOX_REDIRECT_URI` / `MICROSOFT_MAILBOX_REDIRECT_URI`. For plain local dev you don't need these.

VARIABLE	BOTH FILES?	WHAT IT'S FOR
<code>GOOGLE_CLIENT_ID</code> / <code>_SECRET</code>	Yes	Google sign-in and Gmail connect
<code>MICROSOFT_CLIENT_ID</code> / <code>_SECRET</code>	Yes	Microsoft sign-in and Outlook connect
<code>APP_URL</code>	backend only	Base for the mailbox-connect redirect
<code>BETTER_AUTH_URL</code>	frontend only	Base for the sign-in redirect

PART 8 Test it works & fix errors

1 Restart and try sign-in

Restart both services. Open `http://localhost:3000/sign-in` — the "Sign in with Google" and "Sign in with Microsoft" buttons should now complete a real login instead of erroring.

2 Try a mailbox connect

Accounts → connect → **Connect with Google** (or Microsoft). You should reach the provider's consent screen, approve, and land back on the Accounts page with the mailbox added and a WarmupHub folder created.

COMMON OAUTH ERRORS

ERROR	MEANS	FIX
<code>redirect_uri_mismatch</code> (Google)	The URL the app sent isn't registered, to the character.	Copy the two Google redirect URLs from Part 3 exactly — http vs https, trailing slash, port all matter.
"Access blocked: unverified app"	External app in Testing, and this account isn't a test user.	Add the account under OAuth consent screen → Test users, or use Internal, or click "Advanced → continue" for your own test accounts.
<code>invalid_client</code>	Wrong client ID/secret, or not in both env files.	Re-copy both values into backend/.env and frontend/.env.local; restart.
<code>AADSTS50011</code> (Microsoft)	Reply URL mismatch.	Add both Microsoft redirect URLs under Authentication → Web.
<code>AADSTS65001</code> / consent required	Permissions not granted for a company tenant.	An Entra admin grants consent, or use a personal account.
Connects, but can't send/receive later	Token works; the mailbox's SMTP/IMAP is blocked by policy.	Enable authenticated SMTP + IMAP for that mailbox (common on 365).
Google mailbox stops after ~a week	External "Testing" refresh tokens expire in 7 days.	Use Internal for webnco.xyz mailboxes, or reconnect, or publish the app.

PART 9 Reusing one app across many projects

This is the webnco.xyz plan: set up each provider once, reuse everywhere. Here's how to do it cleanly.

1 One client, many redirect URLs

A single Google OAuth client (and a single Azure app) can serve any number of apps — you just add each app's redirect URLs to the same client. For a second project at, say, `http://localhost:4000`, add its `/api/auth/callback/google` and `/api/mailbox-oauth/callback/google` to the existing Google client, and reuse the same client ID and secret. No new project needed.

2 Or one project, many clients

Alternatively keep one Google Cloud project ("WebNCO Dev") and create a separate OAuth client per app inside it. This gives each app its own ID/secret (easier to rotate or revoke one app without affecting others) while sharing one consent screen and one verification status. Pick this if you want isolation between apps; pick step 1 if you want the fewest moving parts.

3 Keep a central credentials note

Record, somewhere private (a password manager or secrets vault — never git), which client ID belongs to which app, its redirect URLs, and where the secret is stored. Future-you setting up the fourth app will thank you.

RULE OF THUMB

For a handful of personal dev apps, **one project + one client + many redirect URLs** is the least work. Move to one-client-per-app only when you want to revoke or rotate apps independently, or when you take one to production and need its own verified consent screen.

EmailWarm — OAuth & Credentials Setup Guide. Redirect paths, scopes and env names were read from the app's source; console screens and menu names reflect the providers' current layouts and may shift over time. Trust the console wording and keep the intent of each step.